Java 26

Table of Contents

1. Getting Started . . . 2
2. Why Upgrade? . . . 3
3. JEPs in Java 26 . . . 3
4. Core . . . 3
   4.1. JEP-500: Prepare to Make Final Mean Final . . . 4
   4.2. JEP-504: Remove the Applet API . . . 5
   4.3. JEP-517: HTTP/3 for the HTTP Client API . . . 5
      4.3.1. Benefits of HTTP/3 . . . 5
      4.3.2. Configuration . . . 6
      4.3.3. Server support and fallback behaviour . . . 6
      4.3.4. When to use HTTP/3 . . . 7
      4.3.5. Limitations . . . 7
   4.4. JEP-525: Structured Concurrency (Sixth Preview) . . . 7
   4.5. JEP-526: Lazy constants (Second preview) . . . 8
   4.6. JEP-529: Vector API (Eleventh incubation) . . . 9
5. HotSpot . . . 10
   5.1. JEP-516: Ahead-of-Time Object Caching with Any GC . . . 10
   5.2. JEP-522: G1 GC: Improve Throughput by Reducing Synchronization . . . 10
6. Language specification . . . 10
   6.1. JEP-530: Primitive Types in Patterns, instanceof, and switch (Fourth Preview) . . . 11
7. Security . . . 11
   7.1. JEP-524: PEM Encodings of Cryptographic Objects (Second preview) . . . 11
8. Migrating . . . 12
   8.1. JEP-504 - Applet API removal . . . 12
   8.2. JDK-8368237 - Thread.stop() removal . . . 12
9. Other changes . . . 13
   9.1. JDK-8329829 - HttpClient: Add a BodyPublishers.ofFileChannel method . . . 13
   9.2. JDK-8334015 - Add support for UUID Version 7 defined in RFC 9562 . . . 13
      9.2.1. Creating and inspecting UUIDv7 . . . 14
      9.2.2. Chronological sorting . . . 14
      9.2.3. Choosing between UUID versions . . . 14
   9.3. JDK-8346944 - Unicode 17 support . . . 15
   9.4. JDK-8342705 - API documentation dark theme . . . 15
   9.5. JDK-8348278 - Reduced default initial heap size . . . 15
   9.6. JDK-8354469 - Enhanced keytool password handling when output is redirected . . . 16
   9.7. JDK-8354548 - Update CLDR to Version 48.0 . . . 16

9.8. JDK-8357653 - Inner classes of type parameters emitted as raw types in signatures 16
9.9. JDK-8362637 - Convert java.nio.ByteOrder to an enum 17
9.10. JDK-8364361 - java.lang.Process should implement Closeable 17
9.11. JDK-8366575 - Remove SDP support 17
9.12. JDK-8369238 - Allow virtual thread preemption on some common class initialization paths 18
9.13. JDK-8369432 - Add Support for JDBC 4.5 MR 18
10. Closing thoughts 18
11. Resources 19

Java 26, the next STS release that's part of the six-month release cadence, will be released on the 17th of March 2026, and once again, it helps move the language forward. The previews that are part of this release help shed some light on what we can expect from Java 27 in September 2026.

Teaser of enhancements

Java 26 gathers ten JEPs across different areas that will enhance performance, security, and modern protocol support, all while steadily progressing major, long-term language projects.

With the addition of HTTP/3, significant optimisations for the G1 garbage collector, and a renewed push for program integrity by restricting final field mutation, this release builds directly on the solid foundation of the JDK 25 LTS release.

1. Getting Started

To try out Java 26:

1. Install it by:
   a. Download from jdk.java.net/26 or use a package manager:
   b. Using a package manager:
      i. SDKman: `sdk install java 26-open`
      ii. Homebrew (macOS): `brew install openjdk@26`
      iii. Chocolatey (Windows): `choco install openjdk26`
2. For preview features, compile and run with `--enable-preview`:

```
javac --enable-preview --release 26 YourFile.java
java --enable-preview YourFile
```

2. Why Upgrade?

- **Free performance**: benefit from improved average application throughput with a leaner G1 garbage collector and faster startup times through ahead-of-time (AOT) object caching that now works with any collector, including the low-latency ZGC.
- **Try out the latest features**: continue working with preview and incubator APIs that are maturing towards becoming permanent fixtures, including Structured Concurrency, pattern matching for primitives, and the high-performance Vector API. Please also remember to provide feedback on the relevant mailing lists.
- **Enhanced security and integrity**: prepare for a future where `final` truly means `final`, enhancing program safety and potential performance. The release also simplifies cryptographic operations with an improved PEM encoding API.
- **Modern protocol support**: stay current with web standards by seamlessly interacting with HTTP/3 servers using the familiar HTTP Client API with minimal code changes.
- **Reduced technical debt**: the final removal of the long-deprecated and obsolete Applet API helps clean up the platform, as it is no longer supported by modern browsers or recent JDK releases

3. JEPs in Java 26

JEP	Title	Status
500	Prepare to Make Final Mean Final	Final
504	Remove the Applet API	Final
516	Ahead-of-Time Object Caching with Any GC	Final
517	HTTP/3 for the HTTP Client API	Final
522	G1 GC: Improve throughput by reducing synchronization	Final
524	PEM Encodings of Cryptographic Objects	Second Preview
525	Structured Concurrency	Sixth Preview
526	Lazy Constants	Second Preview
529	Vector API	Eleventh Incubation
530	Primitive Types in Patterns, instanceof, and switch	Fourth Preview

4. Core

This release brings HTTP/3, which helps keep Java up-to-date with the latest specifications, and we see nice progress on ongoing JEPs such as the Vector API.

4.1. JEP-500: Prepare to Make Final Mean Final

When deep reflection is used to mutate final fields, warnings will be issued. The intent is to help developers prepare for future releases, where this is restricted to help make our programs safer and maybe faster by facilitating further JVM optimisations. (see for reference: integrity by default).

> Please pay attention to this JEP if your framework/library heavily depends on reflection, such as serialisation libraries.

An example of such a warning:

```
WARNING: Final field foo in x.Y has been [mutated/unreflected for mutation] by class
dev.simonverhoeven.z.caller in module Foo (file:/path/to/somefile.jar)
WARNING: Use --enable-final-field-mutation=N to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field
mutation is enabled
```

API developers can avoid the current warnings and future restrictions by selectively enabling final field mutation where it's essential.

To enable final field mutation by any code on the class path, regardless of where the field is declared, we can use the command-line option: `--enable-final-field-mutation=ALL-UNNAMED`.

In case we want to be a bit more fine-grained, we can restrict it to specific modules by using: `--enable-final-field-mutation=module1, module2` Do keep in mind that the to-be-mutated field must also be open to the code that's performing the deep-reflection.

Besides passing in the final-field option directly to the launcher as an option, there are other ways:

- using an argument-file
- setting `JDK_JAVA_OPTIONS`
- adding it to the manifest of an executable JAR
- through the `JNI Invocation API`, passing the option to an embedded JVM that it creates
- passing it to `jlink` when creating a custom Java runtime

We can also control what action the Java runtime takes when an illegal mutation is attempted:

- `--illegal-final-field-mutation=allow` will allow it without warnings
- `--illegal-final-field-mutation=warn` which is the default in JDK26 will allow it with a warning
- `--illegal-final-field-mutation=debug` will behave the same as warn, but will also issue a stack trace
- `--illegal-final-field-mutation=deny` will result in `Field::set` throwing an `IllegalAccessException`. This will become the default in a future release

To help facilitate finding code that mutates final fields, it's recommended to use `--illegal-final-field-mutation=debug` and/or use the Java Flight Recorder and look for `jdk.FinalFieldMutation`

events.

```
$ java -XX:StartFlightRecording:filename=catchmutation.jfr ...
$ jfr print --events jdk.FinalFieldMutation catchmutation.jfr
```

4.2. JEP-504: Remove the Applet API

The `Applet` API was deprecated in JDK 17 through JEP-398. Generally speaking, this will have a negligible impact given that web browsers no longer support applets.

The following will be removed:

- `java.applet` package
- `java.beans.AppletInitializer`
- `javax.swing.JApplet`

Additionally, references to the above will be removed from:

- `java.beans.Beans`
- `javax.swing.RepaintManager`

4.3. JEP-517: HTTP/3 for the HTTP Client API

The HTTP/3 protocol will now be supported by the HTTP Client API, allowing us to easily interact with HTTP/3 servers with minimal changes. The QUIC protocol will not receive an API through this JEP, nor a server-side implementation.

> *HTTP/3*
>
> HTTP/3 uses QUIC over UDP, which is a different transport layer, and non-upgradable from HTTP1.1 or HTTP2 connections, which is the reason we need to explicitly select it.

4.3.1. Benefits of HTTP/3

HTTP/3 brings several improvements over its predecessors:

- **Reduced connection overhead**: eliminates head-of-line blocking at the transport layer
- **Faster connection establishment**: combines transport and cryptographic handshakes
- **Better handling of packet loss**: independent stream handling means loss in one stream doesn't block others
- **Improved performance on lossy networks**: particularly beneficial for mobile connections
- **Better connection migration**: maintains connections when switching networks (e.g., WiFi to cellular)

4.3.2. Configuration

This can be done for all requests by configuring it on the HttpClient object:

```
HttpClient client = HttpClient.newBuilder()
   .version(HttpClient.Version.HTTP_3)
   .build();
```

Or on a request basis:

```
HttpRequest request =
HttpRequest.newBuilder(URI.create("https://github.com/SimonVerhoeven/java26-demo"))
                          .version(HttpClient.Version.HTTP_3)
                          .GET().build();
```

4.3.3. Server support and fallback behaviour

It's important to note that HTTP/3 requires explicit server support. Servers must advertise their HTTP/3 capabilities, typically through the `Alt-Svc` (Alternative Service) HTTP header or DNS records. When you specify `HTTP_3` as the version:

- The client will attempt to connect using HTTP/3 if the server advertises support
- If HTTP/3 is unavailable or the connection fails, the client will **automatically fall back** to HTTP/2 or HTTP/1.1
- This fallback mechanism ensures your application remains functional even when HTTP/3 isn't available

```
HttpClient client = HttpClient.newBuilder()
   .version(HttpClient.Version.HTTP_3)
   .build();

HttpRequest request = HttpRequest.newBuilder()
   .uri(URI.create("https://example.com/api"))
   .build();

// Will use HTTP/3 if available, otherwise falls back to HTTP/2 or HTTP/1.1
HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

// You can check which protocol version was actually used
System.out.println("Protocol used: " + response.version());
```

Testing HTTP/3 Support

Before deploying HTTP/3 in production, verify that your target servers actually support it. You can check HTTP/3 availability using online tools or by examining

> the server's `Alt-Svc` headers. Not all CDNs and hosting providers have enabled HTTP/3 support yet.

4.3.4. When to use HTTP/3

Consider enabling HTTP/3 when:

- Your application communicates with modern CDNs or cloud services that support HTTP/3 (e.g., Cloudflare, Google Cloud, AWS CloudFront)
- Your users experience high-latency or lossy network conditions (mobile applications)
- You need improved performance for API-heavy applications with many concurrent requests
- You want to future-proof your networking stack

However, keep in mind that:

- Corporate networks or firewalls may block UDP traffic, preventing HTTP/3 connections
- Some proxy servers don't yet support HTTP/3
- The automatic fallback ensures compatibility, but you should test in your specific environment

4.3.5. Limitations

- No server-side HTTP/3 implementation is provided by this JEP
- No direct API access to the underlying QUIC protocol
- UDP port 443 must be accessible (some networks block UDP traffic)
- Initial connection may require an additional round trip to discover HTTP/3 support via `Alt-Svc`

4.4. JEP-525: Structured Concurrency (Sixth Preview)

This proposal aims to address some of the frequently encountered challenges in concurrent programming by introducing an API that treats groups of related tasks running in different threads as a single unit of work.

The proposed approach, also known as structured concurrency, will help us streamline error handling, cancellation, and observability, making concurrent code more reliable and easier to manage.

The main target is promoting a style of concurrent programming that eliminates common risks such as thread leaks and cancellation delays, while also improving the observability of concurrent code. The API itself is centered around `StructuredTaskScope`, which allows us to define a clear hierarchy of tasks and subtasks, ensuring that all subtasks are completed or canceled before the parent task exits. This enforces a structured flow of execution, similar to single-threaded code, where subtasks are confined to the lexical scope of their parent task. The API also provides built-in shutdown policies (e.g., `ShutdownOnFailure` and `ShutdownOnSuccess`) to handle common concurrency patterns, such as cancelling all subtasks if one fails or succeeds. We can also define our own shutdown policies.

By reifying the task-subtask relationship at runtime, structured concurrency makes it easier for us to reason about and debug concurrent programs. It also integrates well with observability tools, such as thread dumps, which can now display the hierarchical structure of tasks and subtasks.

This enhancement does not aim to replace existing concurrency constructs like `ExecutorService` or `Future`, but rather to complement them by offering a more structured and reliable alternative for managing concurrent tasks.

Compared to the fifth preview (JEP-505), which saw several API changes such as the public constructors of `StructuredTaskScope` being replaced with static factory methods, the changes are relatively small this time:

- The `allSuccessfulOrThrow()` method on `Joiner` now returns a list of results rather than a stream of subtasks
- The `anySuccessfulResultOrThrow()` method on `Joiner` has been renamed to `anySuccessfulOrThrow`
- The static `open` method of `StructuredTaskScope` that accepted `Joiner` and `Function` now expects `Joiner` and `UnaryOperator`
- A new method `onTimeOut` has been added to the `Joiner` interface, which allows implementations to return a result when the timeout expires

In this example, we're starting three subtasks in parallel with the `StructuredTaskScope` API and waiting until we get the results of all the subtasks to compose the response. If one of the subtasks were to fail, the other, still-running subtasks will be cancelled, and `scope.throwIfFailed()` will throw the exception that happened in the failed subtask.

```
try (final var scope = new StructuredTaskScope.ShutdownOnFailure()) {
   Supplier<Person> personTask =
           scope.fork(() -> findPerson(userId));
   Supplier<Weather> weatherTask =
           scope.fork(this::fetchWeather);
   Supplier<Activity> activityTask =
           scope.fork(() -> findActivity(userId));

   scope.join();
   scope.throwIfFailed();

   final var person = personTask.get();
   final var weather = weatherTask.get();
   final var activity = activityTask.get();

   return new GarderobeSelectionInput(person, weather, activity);
}
```

4.5. JEP-526: Lazy constants (Second preview)

This preview API, previously known as JEP-502: Stable values, introduces immutable value holders that are at most initialised once, as it will help us move towards deferred immutability through

`LazyConstant` and `StableSupplier`.

One of the key benefits of stable values is in how they're handled by the JVM. Internally, the stable values use the internal `@Stable` annotation, which, despite the values being non-final, marks them as not changing after the initial updates. Thanks to this, these values benefit from the same constant-folding optimisations as final-fields.

Compared to the previous iteration, we see quite a number of changes, so please try out this API and provide feedback on the mailing lists.

1. The API has been renamed from `StableValue` to `LazyConstant` to better capture the intended high-level use case of a single constant value that is initialized lazily.
2. The focus of the API has moved to more high-level use cases, so low-level methods such as `orElseSet` and `trySet` have been removed. Meanwhile, the factory methods that take value-computing functions remain.
3. The factory methods for lazy lists and maps have been moved to their respective interfaces to make them easier to discover
4. The API has been simplified by removing the `function` and `intFunction` factory methods, given that they provided minimal benefits over lazy lists and maps.
5. Null is no longer allowed as a computed value to improve performance, and this better aligns lazy constants with other constructs in the language, such as unmodifiable collections and `ScopedValue`

For example, let's define a `LazyConstant` that initialises a `MapData` object by loading it only when it's first accessed:

```
private final LazyConstant<MapData> mapData =
    LazyConstant.of(this::loadMapData); // <= not initialized yet

public Locale getSatelliteLayer() {
   // The first call initializes via loadMapData(); subsequent calls reuse the cached
value
   return mapData.get().getSatelliteLayer();
}
```

4.6. JEP-529: Vector API (Eleventh incubation)

The eleventh iteration of an API for vector computations, this time without any significant changes compared to the previous iteration (JEP-508). It aims to be a way to express these in a manner that compiles reliably to optimal vector instructions on supported CPU architectures to achieve better performance than equivalent scalar computations.

Two important changes are the increased usage of auto-vectorisation on supporting x64 (AVX2/AVX-512) and AArch64 (NEON/SVE) architectures, and now leveraging the foreign function and memory API to link native mathematical-function libraries.

The architects have confirmed that the API will remain in incubation until project Valhalla enters

preview itself so that the API can leverage the expected performance and in-memory representation enhancements. Hence, why it's at its 11th incubation.

Project Valhalla

Valhalla is not part of JDK 26, but the early-access builds are now available for experimentation.

5. HotSpot

This release once again brings us JEPs, which provide free performance upgrades just by upgrading!

5.1. JEP-516: Ahead-of-Time Object Caching with Any GC

The ahead-of-time (AOT) cache introduced by Project Leyden will be enhanced so that it can be used with any garbage collector (GC). This notably includes the low-latency Z Garbage Collector (ZGC), as previously GC-specific formats were used that were not compatible with ZGC's concurrent collection approach.

To achieve this, a neutral, GC-agnostic format will be used to load cached Java objects sequentially into memory, rather than the current approach of directly mapping them into memory in a GC-specific format. This will enable us to combine ZGC's low latency with AOT's fast startup, eliminating the historical tradeoff between startup performance and tail latency.

This change will result in faster startup and warmup times.

5.2. JEP-522: G1 GC: Improve Throughput by Reducing Synchronization

The goal is to increase the average application throughput when the default G1 garbage collector is being used.

This will be achieved by reducing the synchronization overhead between threads and reducing the size of the injected code for write barriers while making sure that the overall architecture of G1 is maintained.

6. Language specification

The language is once again evolving to offer us more flexibility and convenience.

6.1. JEP-530: Primitive Types in Patterns, instanceof, and switch (Fourth Preview)

This JEP, first introduced as JEP-455, aims to enhance pattern matching by allowing primitives in all pattern contexts and allowing one to use them with instanceof and switch as well.

```
if (someObject instanceof int someInt) {
    System.out.println("The int was: " + someInt);
}
```

Compared to the third preview in JDK 25 through JEP-507, there are 2 changes.

1. The definition of unconditional exactness has been enhanced: The concept of "unconditional exactness" determines if a type pattern in a `switch` will always match, making subsequent cases unreachable. JEP 530 refines this definition to be more accurate, which in turn improves the next point
2. Tighter dominance checks have been introduced in switch constructs: Dominance refers to one pattern matching all the values that another pattern matches. With the updated rules, the compiler can now more reliably detect when a `case` label is dominated by a previous one (and is therefore unreachable) and will report a compile-time error

This means that a small number of switch constructs that were legal under JEP-507 will be rejected under JEP-530. For instance, a case with a constant label that can be unconditionally converted to the type of a preceding pattern (like a `case 42` after a `case short s`) will now be seen as dominated and cause a compilation error, preventing a potentially unreachable branch of code.

```
// This was legal in JEP-507 but will fail in JEP-530
switch (someValue) {
    case short s -> System.out.println("Short: " + s);
    case 42 -> System.out.println("Forty-two"); // Now a compile error!
}
```

7. Security

In this release, we see ongoing work to improve cryptographic tooling with a simplified API for PEM encoding, making it easier to work with keys, certificates, and other cryptographic objects.

7.1. JEP-524: PEM Encodings of Cryptographic Objects (Second preview)

The second preview of this API which introduces a **preview API** in Java 26 for encoding/decoding cryptographic objects (keys, certificates, CRLs) to/from the PEM (Privacy-Enhanced Mail) format, simplifying a previously manual and error-prone process.

The API centres on `DEREncodable`, `PEMEncoder`, and `PEMDecoder` classes, supporting standards like `PKCS#8` and `X.509`, with built-in encryption for private keys.

The goals include ease of use and interoperability with tools like `OpenSSL`, thus addressing a gap highlighted by developer surveys.

The design avoids extending legacy APIs such as `KeyFactory` in favour of a dedicated, immutable, and thread-safe solution, though encrypted keys require password handling via `withEncryption()` /`withDecryption()`.

It is possible that future iterations may expand support for non-standard PEM types via the `PEM` class.

As this is a preview feature, you'll need to enable it through `--enable-preview` to experiment with it.

Compared to the previous preview in Java 25 through JEP-470, we see some changes:

- The `PEMRecord` class has been renamed to `PEM` and now offers a `decode()` method to return the decoded Base64 content
- The `EncryptedPrivateKeyInfo` class has been expanded with a `getKeyPair` method that allows us to decrypt a `PKCS#8-encoded` text containing a `PublicKey`
- The `PEMEncoder` and `PEMDecoder` classes now allow us to encrypt and decrypt both `KeyPair` and `PKCS8EncodedKeySpec` objects.

8. Migrating

It's always recommended to move away from deprecated classes and methods as soon as possible, ideally before they're actually removed.

In this release, we'll need to pay attention in case we're still using Applets.

8.1. JEP-504 - Applet API removal

This API has been removed as part of JEP-504, so if you rely on this, you will need to migrate to a different API. For example, the `AWT` API contains several alternatives for the `Applet` class.

If you're casting the result of a `java.net.URLConnection` `getContent()` method to `java.applet.AudioClip`, the code will need to be adapted to the new return type determined by the JDK's built-in content handlers.

8.2. JDK-8368237 - Thread.stop() removal

The `Thread.stop()` method has been removed; it was initially deprecated in Java 1.2, which was released in December 1998. If your code still uses this method, you'll need to refactor to use proper thread interruption mechanisms.

Instead of:

```
thread.stop();
```

Use:

```
thread.interrupt();
```

9. Other changes

Beyond the features delivered through enhancement proposals, there are also some other changes.

> This is just a small subset which piqued my interest.

9.1. JDK-8329829 - HttpClient: Add a BodyPublishers.ofFileChannel method

We can now stream files through `FileChannel` to a `HttpRequest`, which means we no longer need to load a file fully in memory beforehand. This is quite an improvement, especially when dealing with large files.

```
try (FileChannel fileChannel =
        FileChannel.open(Path.of("layer.tga"), StandardOpenOption.READ);
    HttpClient client = HttpClient.newHttpClient()) {

    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://www.simonverhoeven.dev/upload"))
        .POST(BodyPublishers.ofFileChannel(fileChannel, 0, fileChannel.size()))
        .build();

    HttpResponse<Void> response = client.send(request,
HttpResponse.BodyHandlers.discarding());
}
```

9.2. JDK-8334015 - Add support for UUID Version 7 defined in RFC 9562

Java 26 introduces native support for UUID version 7 (UUIDv7), a time-ordered UUID format standardised in RFC 9562. This new variant embeds a Unix Epoch timestamp (with millisecond precision) in its most significant bits, whilst maintaining randomness in the remaining bits to ensure uniqueness. This makes it particularly well-suited for database applications where insertion order and efficient indexing are important considerations.

9.2.1. Creating and inspecting UUIDv7

The new `UUID.ofEpochMillis(long)` method allows us to create a UUIDv7 from a timestamp:

```
// Create a UUIDv7 from the current timestamp
UUID uuid = UUID.ofEpochMillis(System.currentTimeMillis());
System.out.println(uuid); // e.g., 018e9e35-97af-7290-8000-123456789abc

// Each call generates a unique UUID, even with the same timestamp
UUID another = UUID.ofEpochMillis(System.currentTimeMillis());
System.out.println(uuid.equals(another)); // false - randomness ensures uniqueness

// Retrieve the embedded timestamp
long timestamp = uuid.getEpochMillis();
Instant creationTime = Instant.ofEpochMilli(timestamp);
System.out.println("UUID created at: " + creationTime);

// Verify the UUID version
System.out.println("Version: " + uuid.version()); // 7
```

9.2.2. Chronological sorting

UUIDv7's time-ordered nature enables natural chronological sorting:

```
UUID earlier = UUID.ofEpochMillis(Instant.parse("2026-03-
17T09:00:00Z").toEpochMilli());
UUID later = UUID.ofEpochMillis(Instant.parse("2026-03-17T10:00:00Z").toEpochMilli());

System.out.println(earlier.compareTo(later) < 0); // true - naturally time-ordered

// This makes UUIDv7 ideal for:
// - Range queries (WHERE id BETWEEN ? AND ?)
// - Pagination with cursor-based approaches
// - Time-based partitioning
```

9.2.3. Choosing between UUID versions

UUIDv7 **complements** the existing v4 format rather than replacing it. The choice between them depends on your requirements:

- Use **v4** if you need simplicity, maximum randomness, and broad compatibility are required, and insertion order or time ordering is unimportant.
- Use **v7** if you need insertion order, lexicographical sorting, more efficient database indexing, or want to embed creation time in the identifier.

Backward Compatibility

UUIDv7 is fully compatible with the existing UUID infrastructure. All standard

UUID methods (`toString()`, `fromString()`, etc.) work seamlessly, and UUIDv7 can be stored in existing UUID database columns without schema changes.

Database Performance Benefits

UUIDv7's time-ordered nature can significantly reduce database index fragmentation compared to random UUIDs, leading to better insertion performance in high-throughput scenarios. In databases with B-tree indices (such as PostgreSQL, MySQL InnoDB, and Oracle Database), time-ordered UUIDs substantially reduce index page splits compared to random UUIDs in sequential insertion scenarios, leading to both improved write throughput and reduced storage overhead. Actual performance gains depend on workload patterns, database configuration, and hardware characteristics.

This makes UUIDv7 particularly valuable for:

- Logging systems with high volume writes
- Event sourcing applications
- Time-series data storage
- Distributed systems requiring ordered identifiers
- Applications where B-tree index efficiency is critical

9.3. JDK-8346944 - Unicode 17 support

Java 26 adds support for Unicode 17.0, which includes new characters, scripts, and emoji. This ensures Java stays current with the latest text encoding standards.

This release includes 4,803 new characters, of which 7 are new emoji code points, 4 are new scripts, and technical fixes.

9.4. JDK-8342705 - API documentation dark theme

Java 26 introduces a dark theme for JavaDoc, making it easier on the eyes for developers who prefer dark mode. This theme can be toggled in the documentation's header.

9.5. JDK-8348278 - Reduced default initial heap size

The default initial heap size has been reduced when neither `Xms` nor `-XX:InitialRAMPercentage` has been used. Prior to this release, the heap was set to 1/64th of the physical RAM by default; now it's set to 1/500th. To put this into context, on a 32 GiB this resulted in a 512 MiB heap, whereas in Java 26 it will be 64 MiB.

Not only does this represent a significant reduction in the memory consumption of our applications, but it can also speed up the startup of our applications given that the initial data structure the Garbage Collector has to create will be smaller.

9.6. JDK-8354469 - Enhanced keytool password handling when output is redirected

This command reads passwords from the system console to avoid them being displayed on the screen. However, in previous releases, this did not work when the input or output stream was redirected. As of this release, the password will no longer be displayed on the screen even if the standard output stream is redirected. This improvement has also been made to the `jarsigner` command and the JAAS `TextCallbackHandler` API.

9.7. JDK-8354548 - Update CLDR to Version 48.0

This updates the Unicode Common Locale Data Repository (CLDR) in the JDK to CLDR version 48.

This entails:

- CLDR-16439 zone alias South_Pole was moved to McMurdo, so remove it from Auckland (#4592)
- CLDR-16665 key/type core values (#4577)
- CLDR-13688 Correct the first day of the week for Iceland from Monday to Sunday (#4379)
- CLDR-18464 Add a Before Hijrah era to the Islamic calendars (#4581)
- CLDR-18572 Add more European Englishes (#4645)
- CLDR-18253 Add en_JP (#4634)
- CLDR-18603 Add GyM, GyMEd availableFormats to root, en (#4679)
- CLDR-18538 root,en: change H patterns without m[s] to add literal 'h' (#4708)
- CLDR-9814 Recover Turkey metazone names (#4741)
- CLDR-18788 Fix Russian metazones (#4833)
- CLDR-13542 Fix Swiss grouping separator (#5019)

9.8. JDK-8357653 - Inner classes of type parameters emitted as raw types in signatures

This resolves an issue where `javac` false believes that the inner class of type parameters returns just `Object`. Now the correct type is being emitted.

```
class Demonstration {
    static abstract class Getters<T> {
        abstract class Getter {
            abstract T get();
        }
    }

    static class GettersUsage<T, G extends Getters<T>> {
        public T test(G.Getter getter) {
            return getter.get();
```

```
        }
    }
}
```

9.9. JDK-8362637 - Convert java.nio.ByteOrder to an enum

This class, which predates the addition of enum to Java, has been converted to an enum:

```
public enum ByteOrder {
    LITTLE_ENDIAN,
    BIG_ENDIAN;

    private static final ByteOrder NATIVE_ORDER = Unsafe.getUnsafe().isBigEndian() ?
BIG_ENDIAN : LITTLE_ENDIAN;

    public static ByteOrder nativeOrder() {
        return NATIVE_ORDER;
    }
}
```

9.10. JDK-8364361 - java.lang.Process should implement Closeable

When a process launched via ProcessBuilder is torn down, the caller bears responsibility for closing its streams and ensuring the process terminates properly. Introducing a `Process.close()` method would provide a straightforward and intuitive means of ensuring all streams are closed, and the process is terminated.

The try-with-resources statement is commonly used to open streams whilst guaranteeing they're closed upon exiting the block. By implementing `AutoCloseable.close()`, the completeness of stream closure and process termination can be handled elegantly through try-with-resources.

The `close()` method would perform the following actions: close each associated stream and destroy the process should it not have already terminated.

9.11. JDK-8366575 - Remove SDP support

The support for Sockets Direct Protocol, a protocol that has been obsolete for over a decade and cannot be used on modern systems, has been removed from the JDK.

9.12. JDK-8369238 - Allow virtual thread preemption on some common class initialization paths

This update changes how the JVM manages these waiting states. By leveraging the same mechanisms used for `ObjectMonitor` (the internal logic behind synchronised blocks), the JVM can now preempt or unmount virtual threads that are waiting for class initialisation. Thus, further reducing when thread pinning might occur.

This means:

- Improved Robustness: Virtual threads waiting on invokestatic, new, or static field access will no longer hog carrier threads.
- Deadlock mitigation: The specific "pathological" deadlock described above is largely eliminated because the carrier thread is freed to do other work, such as finishing the very initialisation the virtual thread is waiting for.
- Seamless scalability: You no longer need to worry as much about "heavy" class initialisers (static blocks that take a long time or perform complex logic), causing a ripple effect that stalls your entire virtual thread pool.

9.13. JDK-8369432 - Add Support for JDBC 4.5 MR

This update to the specification includes the following changes:

- Deprecate `SQLPermission` for removal
- Enhance the `Blob`/`Clob`/`Array`/`SQLXML`/`NClob` interfaces to extend/support AutoClosable
- Add the SQL types `DECFLOAT`, `JSON` to `Types.Java` and `JDBCType.java`
- Add the quoted identifier methods that were added to the `Statement` interface in JDK 9 to the `Connection` interface with the exact same wording and methods
- Clarify the `Timestamp::hashCode` method, which incorrectly indicates that nanos are not used when calculating the hash

10. Closing thoughts

Whilst this release encompasses ten JEPs, it's worth noting that it includes JEP-517: HTTP/3 for the HTTP Client API, which represents one of the most substantial pull requests in recent years. Additionally, the first project Valhalla early-access build has been made available for experimentation.

Beyond the headline features, there are numerous compelling reasons to upgrade. Performance improvements such as the reduced initial heap size and enhanced virtual thread preemption, alongside Unicode 17 support, type 7 UUID creation, and ongoing security enhancements, all contribute to a more robust platform.

This release also resolves a notable issue where inner classes of type parameters were being emitted as raw types (JDK-8357653).

Furthermore, two new Java Flight Recorder events have been introduced to provide deeper insights into application behaviour:

- `StringDeduplication` - monitors string deduplication activity
- `FinalFieldMutation` - tracks final field mutation attempts

These enhancements, whilst perhaps less prominent than the major JEPs, collectively provide tangible benefits simply through upgrading to Java 26.

11. Resources

Some useful resources to dive deeper into the Java ecosystem and stay up to date are:

- GitHub repository - this article's code
- The release notes - The official source for all changes, including new features, bug fixes, and deprecations
- OpenJDK Quality Group - A group of developers that helps ensure the quality of OpenJDK projects and FOSS in general through initiatives such as promoting the testing of projects on Early Access (EA) builds
- The Java version almanack - A great resource with details on distributions, and API differences between various releases
- Foojay - A magnificent Java community offering articles, tutorials, and discussions on the latest in the Java ecosystem
- SDKman! - a great tool to manage the installation of various tools and languages
- Inside Java - News updates by Java team members at Oracle
- Java Community Process - the place where people can propose, discuss, and approve new features through a Java Specification Request (JSR)

© 2026 Simon Verhoeven. Licensed under the MIT License.
Last updated: March 17, 2026